

Universidad La Salle - Arequipa
Facultad de Ingeniería
Escuela Profesional de Ingeniería de Software

Documento Técnico (MTD)

AuthPolicyAnalyzer

Analizador de Fortaleza de Políticas de Autenticación

Arquitectura, diagramas, implementación e interfaz

Curso: Ciberseguridad
Semestre: X
Docente: Marco Antonio Camacho Alatriza
Estudiante: Josué Carlos Alberto Ramos Suyoc

Arequipa, Perú
14 de julio de 2026

Índice

1	Objetivo	2
2	Título del Software	2
3	Tecnologías Utilizadas	2
4	Requisitos Funcionales	2
5	Diagramas del Sistema	3
5.1	Diagrama de Componentes	3
5.2	Diagrama de Clases	4
5.2.1	Módulo <code>core</code> (lógica de dominio)	4
5.2.2	Módulos <code>data</code> , <code>services</code> y <code>reports</code>	5
5.3	Diagramas de Flujo	5
5.3.1	Flujo: Análisis de una Contraseña	6
5.3.2	Flujo: Ataque de Diccionario Inverso	7
5.3.3	Flujo: Verificación HIBP con k-anonimato	8
5.3.4	Flujo: Procesamiento en Lote (CSV)	8
5.4	Diagramas de Secuencia	9
5.4.1	Secuencia: Análisis Individual con HIBP asíncrono	9
5.4.2	Secuencia: Procesamiento de Lote CSV	10
6	Implementación	10
6.1	Estructura del Proyecto	10
6.2	Fragmentos de Código Relevantes	11
6.2.1	Cálculo de la Entropía	11
6.2.2	Ataque de Diccionario Inverso	11
6.2.3	Verificación HIBP con k-anonimato	12
6.2.4	Verificación de la Política	12
7	Interfaz de Usuario	13
7.1	Pestaña Analizar	13
7.2	Pestaña Lote CSV	13
7.3	Pestaña Historial	14
7.4	Pestaña Configuración	15

1. Objetivo

Desarrollar una herramienta de escritorio que evalúe la fortaleza de contraseñas, verifique su exposición en filtraciones públicas y permita configurar y auditar políticas de autenticación, con un diseño orientado a la privacidad: ninguna contraseña se almacena en texto plano ni sale del equipo del usuario.

2. Título del Software

AuthPolicyAnalyzer — Analizador de Fortaleza de Políticas de Autenticación (*Authentication Policy Strength Analyzer*).

3. Tecnologías Utilizadas

El proyecto se ha desarrollado con las siguientes tecnologías:

- **Lenguaje:** Python 3.9+.
- **Interfaz gráfica:** PyQt5 (aplicación de escritorio nativa).
- **Evaluación de fortaleza:** zxcvbn (estimación de fuerza real por patrones).
- **Verificación de filtraciones:** API *Have I Been Pwned* (HIBP) mediante `requests` y el protocolo de *k-anonimato*.
- **Generación de reportes:** `reportlab` (PDF) y la librería estándar `json`.
- **Persistencia:** SQLite (módulo `sqlite3` de la biblioteca estándar, en modo WAL).

4. Requisitos Funcionales

El sistema AuthPolicyAnalyzer cumple con los siguientes requisitos funcionales:

RF-01

Evaluar la fortaleza de una contraseña por longitud, composición y entropía real (zxcvbn).

RF-02

Verificar la exposición en filtraciones públicas (HIBP) usando k-anonimato.

RF-03

Simular un ataque de diccionario con variantes comunes, incluido *leetspeak*.

RF-04

Calcular un puntaje de fortaleza de 0 a 4 con representación visual coloreada.

RF-05

Generar recomendaciones automáticas priorizadas en el idioma activo.

RF-06

Verificar el cumplimiento de una política de autenticación configurable y persistente.

RF-07

Procesar contraseñas en lote a partir de archivos CSV.

RF-08

Generar reportes de evaluación en formato PDF.

RF-09

Exportar resultados individuales y de lote en formato JSON.

RF-10

Mantener un historial de evaluaciones en SQLite (solo hash SHA-256, nunca texto plano).

5. Diagramas del Sistema

Los siguientes diagramas representan la arquitectura, la estructura y el funcionamiento del sistema.

5.1. Diagrama de Componentes

La arquitectura sigue un patrón por capas (*Layered Architecture*) con bajo acoplamiento y alta cohesión. Se distinguen seis paquetes y las librerías externas:

- **ui** (presentación): ventana principal PyQt5, hoja de estilos, internacionalización (i18n) y *widgets* (barra de fortaleza y *workers* de hilos). No contiene lógica de negocio.
- **core** (lógica de dominio): motor de análisis (**analyzer**), criterios de política (**criteria**), ataque de diccionario (**dictionary_attack**) y recomendaciones (**recommendations**). No depende de la UI ni de la base de datos.
- **data** (persistencia): **DatabaseManager** sobre SQLite y el diccionario de contraseñas comunes.
- **services** (servicios externos): cliente de la API HIBP con degradación elegante.
- **batch** (procesamiento masivo): procesador de archivos CSV.
- **reports** (artefactos): generación de PDF y exportación JSON.

Las **librerías externas** (PyQt5, zxcvbn, requests, reportlab) proporcionan la interfaz, la evaluación, la conectividad y la generación de reportes.

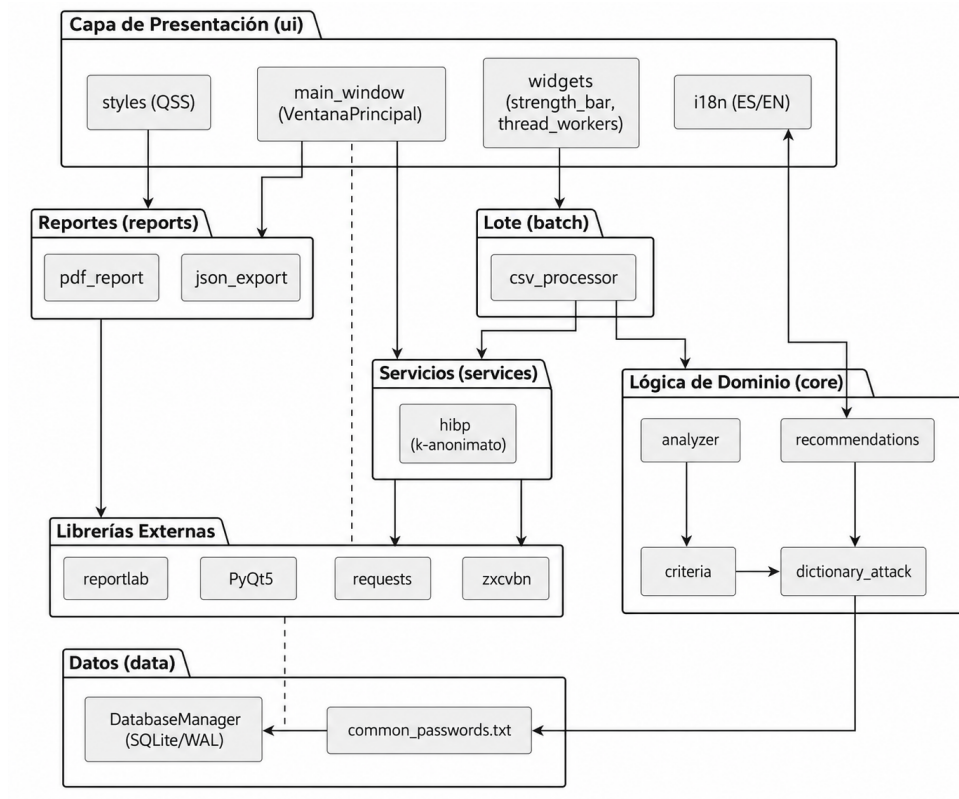


Figura 1: Diagrama de Componentes del Sistema

Fuente: Elaboración propia

5.2. Diagrama de Clases

5.2.1. Módulo core (lógica de dominio)

El módulo `core` contiene la clase de datos `ResultadoAnálisis` (*dataclass* serializable con todas las métricas del análisis) y la clase `Criterios`, que representa la política configurable y expone `verificar()` para contrastar un resultado contra la política. Las funciones `analizar()`, `calcular_entropia()` y `simular_ataque()` orquestan el *pipeline* de evaluación, mientras que `generar_recomendaciones()` aplica un motor de reglas por severidad.

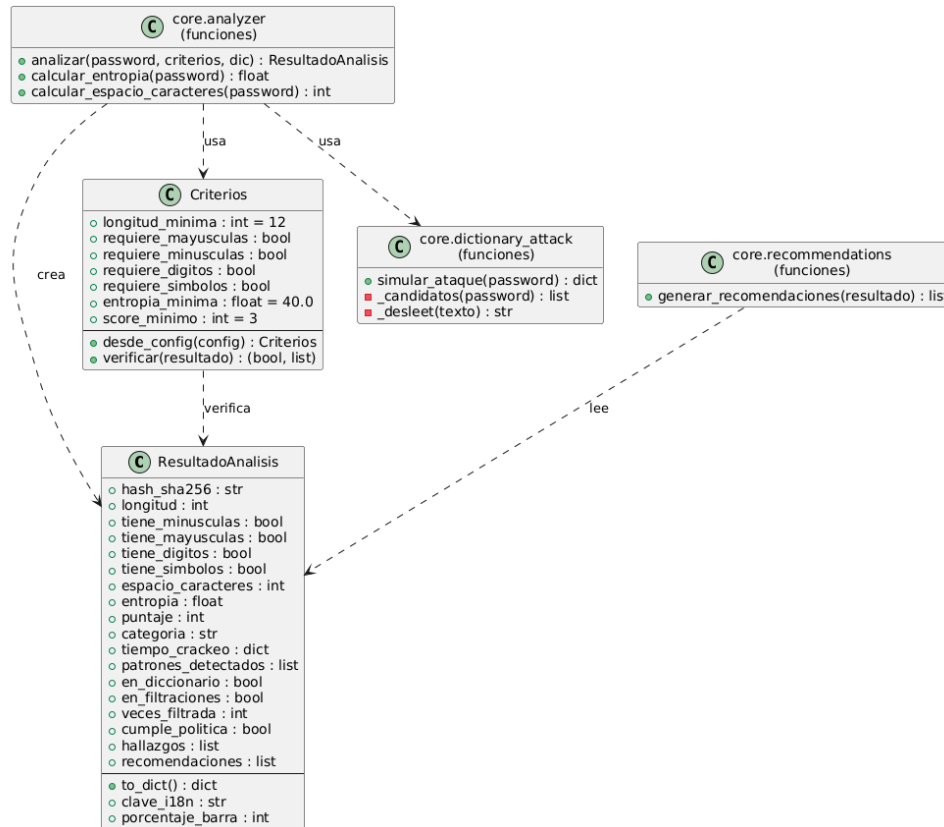


Figura 2: Diagrama de Clases — Módulo core

Fuente: Elaboración propia

5.2.2. Módulos data, services y reports

El módulo **data** expone **DatabaseManager**, con métodos para inicializar el esquema, guardar y consultar evaluaciones y administrar la configuración. El módulo **services** contiene el cliente **hibp** con **verificar()** (k-anonimato y caché de prefijos). El módulo **reports** agrupa **generar_pdf()** y las funciones de exportación JSON (**exportar_resultado** y **exportar_lote**).

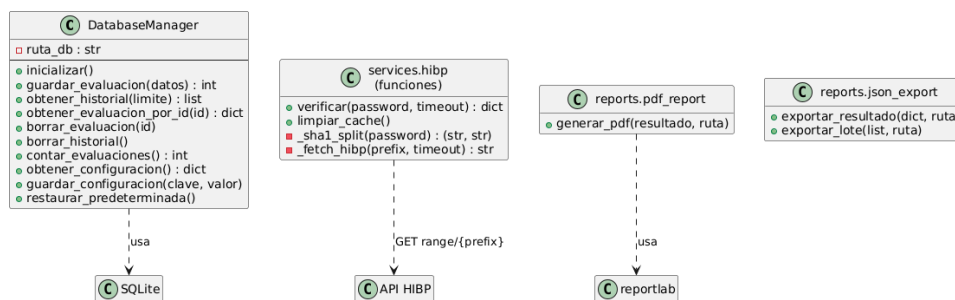


Figura 3: Diagrama de Clases — Módulos data, services y reports

Fuente: Elaboración propia

5.3. Diagramas de Flujo

5.3.1. Flujo: Análisis de una Contraseña

El proceso inicia cuando el usuario ingresa una contraseña. El sistema calcula su hash SHA-256 (sin almacenar la contraseña), determina la composición de caracteres y la entropía teórica, obtiene el *score* y los patrones con zxcvbn, ejecuta el ataque de diccionario inverso y verifica el cumplimiento de la política. En paralelo, un hilo consulta HIBP. Con toda la información se generan los hallazgos técnicos y las recomendaciones, se guarda la evaluación y se muestran los resultados.

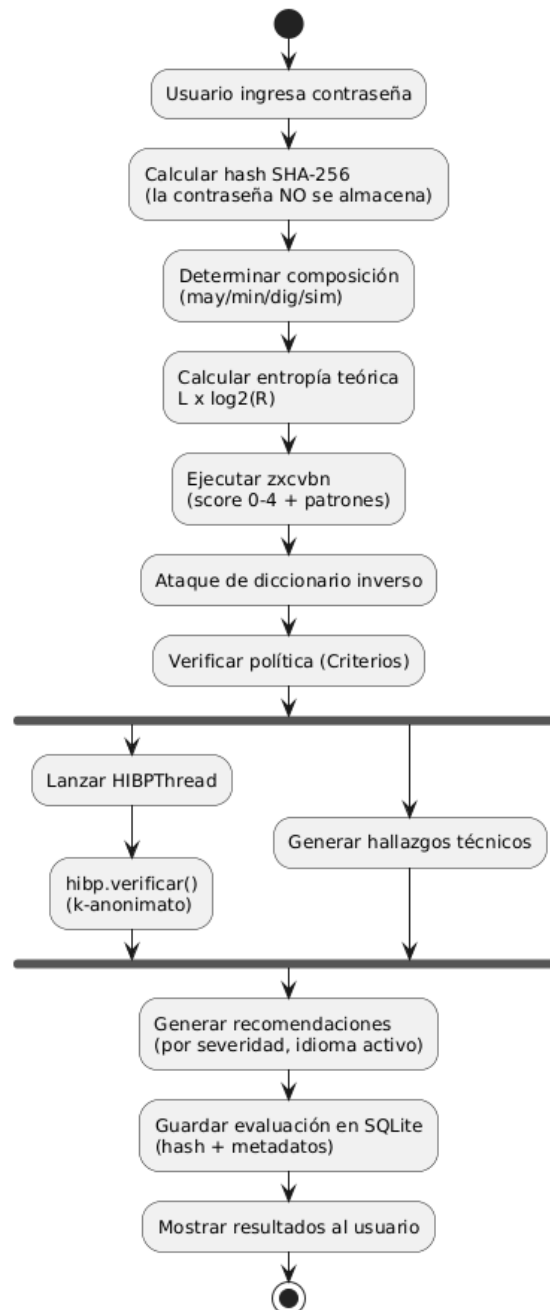


Figura 4: Diagrama de Flujo: Análisis de una Contraseña

Fuente: Elaboración propia

5.3.2. Flujo: Ataque de Diccionario Inverso

En lugar de generar millones de variantes del diccionario (coste exponencial), el sistema aplica transformaciones *inversas* a la contraseña: a minúsculas, a mayúsculas, primera letra mayúscula, reversión de *leetspeak* ($@ \rightarrow a$, $3 \rightarrow e$, $0 \rightarrow o \dots$) y eliminación de prefijos/sufijos numéricos o simbólicos. Cada variante se busca en un conjunto (*set*) cargado en memoria, con complejidad $O(1)$. Si alguna coincide, la contraseña se marca como encontrada, indicando la variante y el esfuerzo estimado.

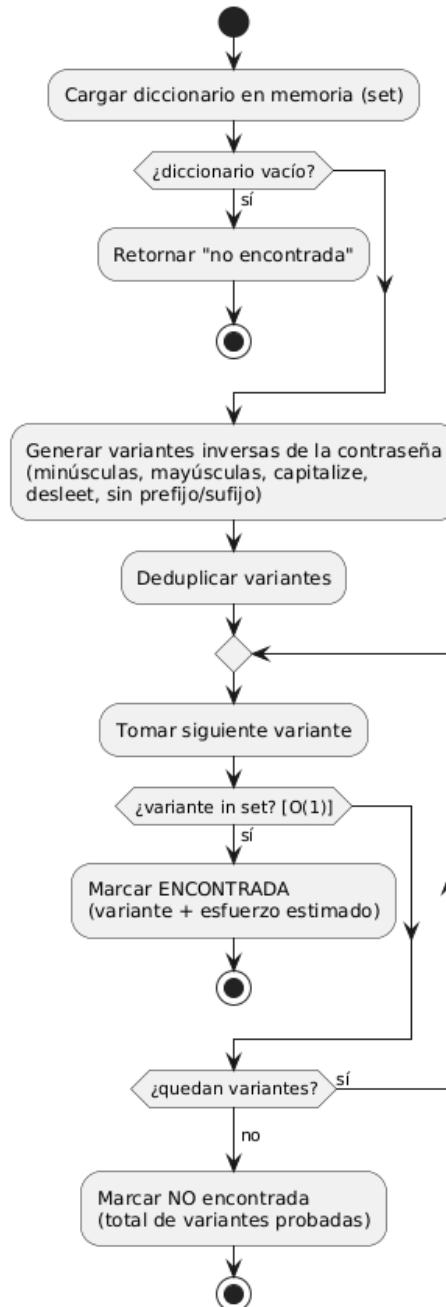


Figura 5: Diagrama de Flujo: Ataque de Diccionario Inverso

Fuente: Elaboración propia

5.3.3. Flujo: Verificación HIBP con k-anonimato

Se calcula el hash SHA-1 de la contraseña de forma local. Se envían únicamente los **primeros 5 caracteres** (prefijo) a la API. Esta devuelve todos los sufijos que comparten ese prefijo junto con su conteo de filtraciones. La comparación final se realiza localmente. Si no hay conexión, la función retorna `disponible = False` sin lanzar excepciones (degradación elegante).

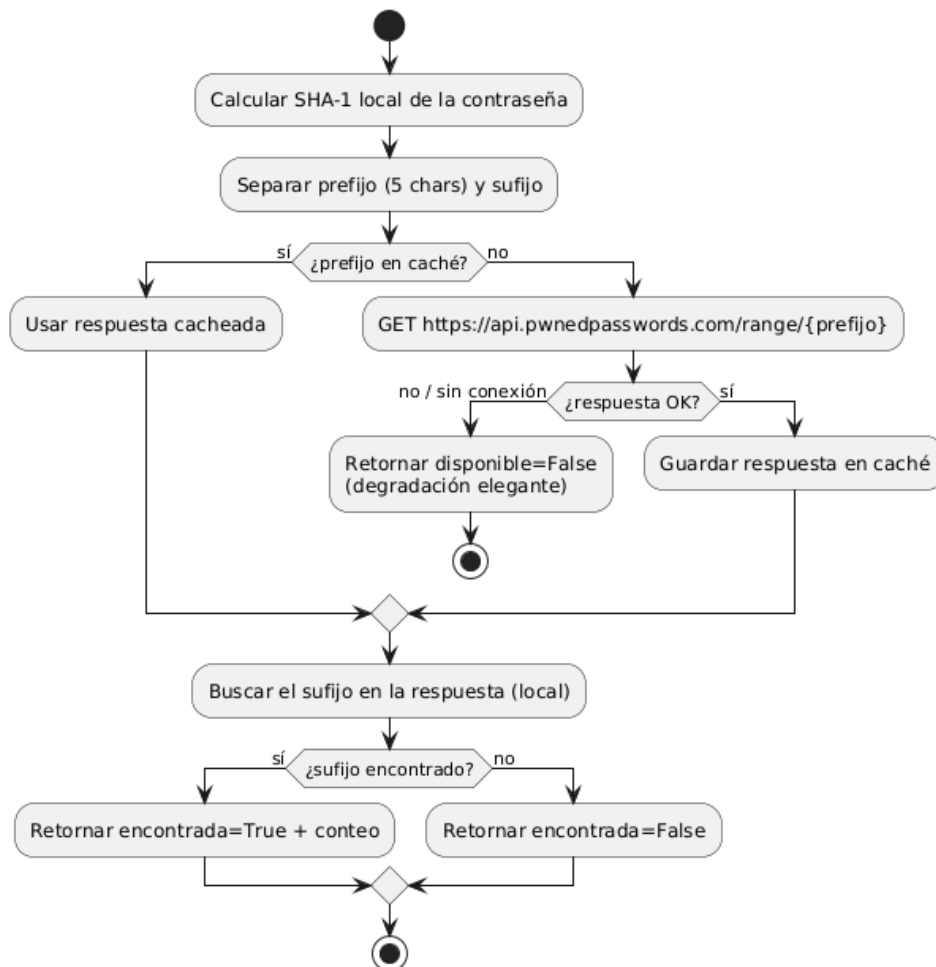


Figura 6: Diagrama de Flujo: Verificación HIBP (k-anonimato)

Fuente: Elaboración propia

5.3.4. Flujo: Procesamiento en Lote (CSV)

El sistema lee el archivo, detecta el delimitador y la cabecera, y resuelve las columnas de usuario y contraseña por nombre (o por posición). Por cada fila válida ejecuta el mismo *pipeline* de análisis (con HIBP opcional y caché de prefijos) y acumula los resultados. Al finalizar calcula un resumen estadístico por categoría, comprometidas, cumplimiento de política y coincidencias de diccionario.

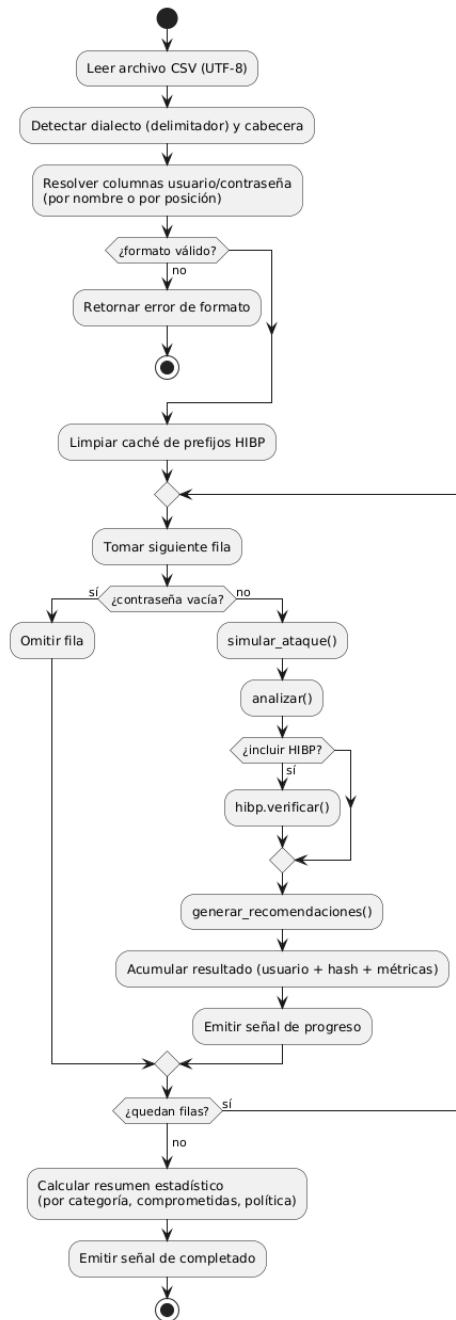


Figura 7: Diagrama de Flujo: Procesamiento en Lote

Fuente: Elaboración propia

5.4. Diagramas de Secuencia

5.4.1. Secuencia: Análisis Individual con HIBP asíncrono

El diagrama muestra la interacción entre el Usuario, la **VentanaPrincipal**, el motor `analizar()`, `simular_ataque()`, `generar_recomendaciones()`, el `HIBPThread` y el `DatabaseManager`. Al pulsar **ANALIZAR**, la ventana ejecuta el análisis local y muestra resultados de inmediato; en paralelo lanza un hilo que consulta HIBP y, al recibir la respuesta, actualiza la interfaz, regenera las recomendaciones y guarda la evaluación en la base de datos.

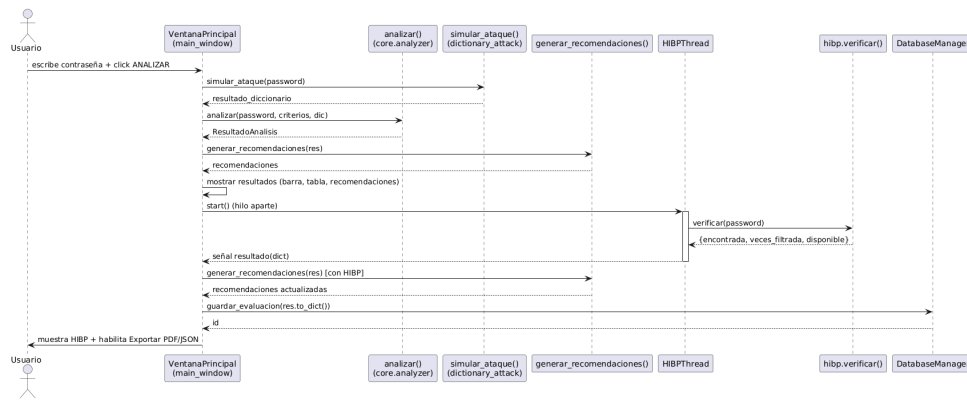


Figura 8: Diagrama de Secuencia: Análisis Individual

Fuente: Elaboración propia

5.4.2. Secuencia: Procesamiento de Lote CSV

Al cargar un CSV, la ventana inicia un `LoteThread` que invoca `procesar_csv()`. Este recorre las filas llamando por cada una a `simular_ataque()`, `analizar()`, `hibp.verificar()` y `generar_recomendaciones()` emitiendo señales de progreso hacia la barra de la interfaz. Al terminar, emite la señal de completado con los resultados y el resumen, que la ventana vuelca en la tabla.

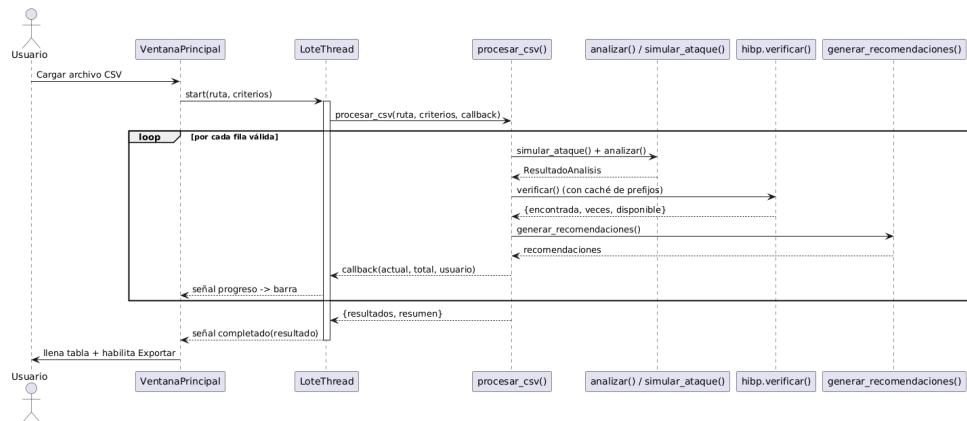


Figura 9: Diagrama de Secuencia: Procesamiento de Lote

Fuente: Elaboración propia

6. Implementación

6.1. Estructura del Proyecto

El proyecto se organiza en una estructura modular que separa claramente las responsabilidades:

```
authpolicyanalyzer/
  main.py           # Punto de entrada (QApplication)
  requirements.txt  # Dependencias del proyecto
  ejemplo_lote.csv  # CSV de muestra
  core/
    analyzer.py     # Logica de dominio
    criteria.py     # Pipeline principal, ResultadoAnálisis
                  # Criterios configurables de politica
```

```

dictionary_attack.py      # Simulador de ataque de diccionario
recommendations.py        # Generador de recomendaciones
data/                     # Capa de datos
    database.py           # DatabaseManager (SQLite, WAL)
    common_passwords.txt  # Diccionario de contraseñas comunes
services/                 # Servicios externos
    hibp.py               # API Have I Been Pwned (k-anonimato)
reports/                  # Generación de reportes
    pdf_report.py         # Reporte PDF (ReportLab)
    json_export.py        # Exportación JSON
batch/                    # Procesamiento en lote
    csv_processor.py      # Procesador de archivos CSV
ui/                        # Interfaz gráfica
    main_window.py        # VentanaPrincipal (PyQt5)
    styles.py             # Hoja de estilos (QSS)
    i18n.py               # Internacionalización ES/EN
widgets/
    strength_bar.py       # Barra de fortaleza
    thread_workers.py     # Workers QThread (HIBP, CSV)

```

6.2. Fragmentos de Código Relevantes

6.2.1. Cálculo de la Entropía

La entropía teórica se calcula a partir del espacio de caracteres presente y la longitud de la contraseña, mediante la fórmula $\text{Entropía} = L \times \log_2(R)$, donde L es la longitud y R el tamaño del espacio de caracteres.

```

1 def calcular_espacio_caracteres(password: str) -> int:
2     """Tamaño del espacio de búsqueda según los tipos de carácter."""
3     espacio = 0
4     if any(c.islower() for c in password):
5         espacio += 26 # a-z
6     if any(c.isupper() for c in password):
7         espacio += 26 # A-Z
8     if any(c.isdigit() for c in password):
9         espacio += 10 # 0-9
10    if any(not c.isalnum() for c in password):
11        espacio += 33 # símbolos imprimibles ASCII
12    return max(espacio, 1)
13
14 def calcular_entropia(password: str) -> float:
15     """Entropía teórica en bits = len × log2(espacio de caracteres)."""
16     espacio = calcular_espacio_caracteres(password)
17     return len(password) * math.log2(espacio)

```

6.2.2. Ataque de Diccionario Inverso

En vez de expandir el diccionario, se generan pocas variantes *inversas* de la contraseña y se buscan en un conjunto en memoria (búsqueda $O(1)$).

```

1 def simular_ataque(password: str) -> dict:
2     """Comprueba si la contraseña (o una derivación común) está en el diccionario."""
3     dic = _cargar_diccionario() # set en memoria (cacheado)
4     variantes = _candidatos(password) # minúsculas, desleet, sin sufijo...
5
6     if not dic:
7         return {"encontrada": False, "variante": "", "descripcion": "",
8             "esfuerzo": "", "total_variantes_probadas": 0}
9

```

```

10 for clave, desc, esf in variantes:
11     if clave in dic:                                # coincidencia -> vulnerable
12         return {"encontrada": True, "variante": clave, "descripcion": desc,
13                 "esfuerzo": esf, "total_variantes_probadas": len(variantes)}
14
15 return {"encontrada": False, "variante": "", "descripcion": "",
16         "esfuerzo": "", "total_variantes_probadas": len(variantes)}

```

6.2.3. Verificación HIBP con k-anonimato

Solo el prefijo de 5 caracteres del hash SHA-1 se envía a la API; la comparación final es local. La contraseña y el hash completo nunca salen del sistema.

```

1 def _sha1_split(password: str) -> tuple[str, str]:
2     """Devuelve (prefijo_5_chars, sufijo) del hash SHA-1 en mayúsculas."""
3     h = hashlib.sha1(password.encode("utf-8")).hexdigest().upper()
4     return h[:5], h[5:]
5
6 def verificar(password: str, timeout: float = 5.0) -> dict:
7     prefix, sufijo = _sha1_split(password)
8     if prefix not in _cache:
9         texto = _fetch_hibp(prefix, timeout)          # GET .../range/{prefix}
10        if texto is None:                             # sin conexión -> degrada
11            return {"encontrada": False, "veces_filtrada": 0, "disponible": False}
12        _cache[prefix] = texto                         # caché de prefijos
13        conteo = _buscar_en_respuesta(_cache[prefix], sufijo) # comparación local
14        if conteo > 0:
15            return {"encontrada": True, "veces_filtrada": conteo, "disponible": True}
16        return {"encontrada": False, "veces_filtrada": 0, "disponible": True}

```

6.2.4. Verificación de la Política

La clase Criterios contrasta un ResultadoAnálisis contra la política activa y devuelve la lista de incumplimientos.

```

1 def verificar(self, resultado) -> tuple[bool, list[str]]:
2     """Devuelve (cumple, incumplimientos)."""
3     inc: list[str] = []
4     if resultado.longitud < self.longitud_minima:
5         inc.append(f"Longitud mínima {self.longitud_minima} (actual: {resultado.longitud})")
6     if self.requiere_mayusculas and not resultado.tiene_mayusculas:
7         inc.append("Falta letra mayúscula")
8     if self.requiere_digitos and not resultado.tiene_digitos:
9         inc.append("Falta dígito numérico")
10    if resultado.entropia < self.entropia_minima:
11        inc.append(f"Entropía mínima {self.entropia_minima:.1f} bits")
12    if resultado.puntaje < self.score_minimo:
13        inc.append(f"Score mínimo {self.score_minimo} (actual: {resultado.puntaje})")
14    return len(inc) == 0, inc

```

Privacidad y Seguridad

En todo el sistema, la contraseña se convierte a hash SHA-256 al inicio del análisis y **jamás** se persiste en texto plano. La base de datos guarda únicamente el hash y los metadatos; los reportes muestran solo la longitud a modo de máscara.

7. Interfaz de Usuario

La aplicación presenta una interfaz de escritorio con un encabezado fijo y cuatro pestañas especializadas, con un estilo visual *neobrutalista* (bordes sólidos, colores planos y tipografía en negrita).

7.1. Pestaña Analizar

Permite evaluar una contraseña individual y muestra la barra de fortaleza, las métricas (entropía, espacio de caracteres, puntaje), la tabla de detalles, el estado de la verificación HIBP y las recomendaciones, con botones para exportar a PDF y JSON.

The screenshot shows the 'AuthPolicyAnalyzer' application interface, specifically the 'Analyze' tab. The header includes the application name and a language selector set to 'EN'. The main navigation bar contains four tabs: 'Analyze' (active), 'CSV Batch', 'History', and 'Settings'. The 'Analyze' section is titled 'Password to Evaluate' and features a text input field containing 'password', an 'Ocultar' button, and an 'ANALYZE' button. Below the input field is a strength bar labeled 'VERY WEAK (0 / 4)' with a red progress indicator. Three metrics are displayed: 'Entropy: 37.60 bits', 'Espacio: 26 chars', and 'zxcvbn score: 0/4'. The 'Analysis Details' section contains a table with the following data:

Online attack (throttled)	2 minutes
Online attack (no throttle)	less than a second
Offline — slow hash	less than a second
Offline — fast hash	less than a second

The 'Breaches (HIBP)' section shows '52,372,427 times in breaches' in red text. At the bottom right are 'Export PDF' and 'Export JSON' buttons. The footer indicates 'Total: 3 records'.

Figura 10: Interfaz — Pestaña Analizar

Fuente: Elaboración propia

7.2. Pestaña Lote CSV

Permite cargar un archivo CSV, muestra la barra de progreso durante el procesamiento y presenta los resultados en una tabla con opción de exportarlos.

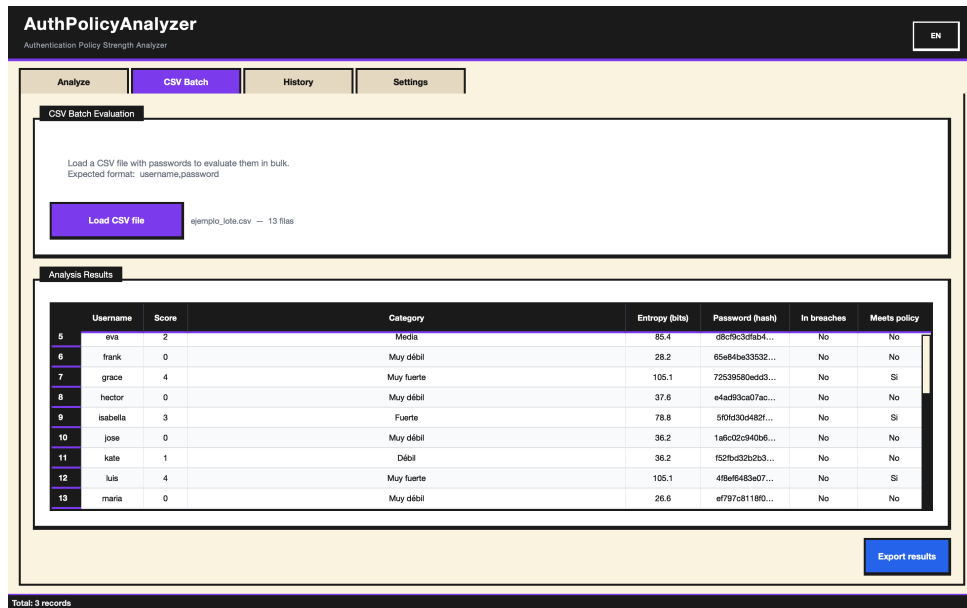


Figura 11: Interfaz — Pestaña Lote CSV

Fuente: Elaboración propia

7.3. Pestaña Historial

Lista las evaluaciones almacenadas en la base de datos local, con columnas de ID, fecha, longitud, puntaje, categoría, entropía, filtración y política, y acciones para borrar registros.

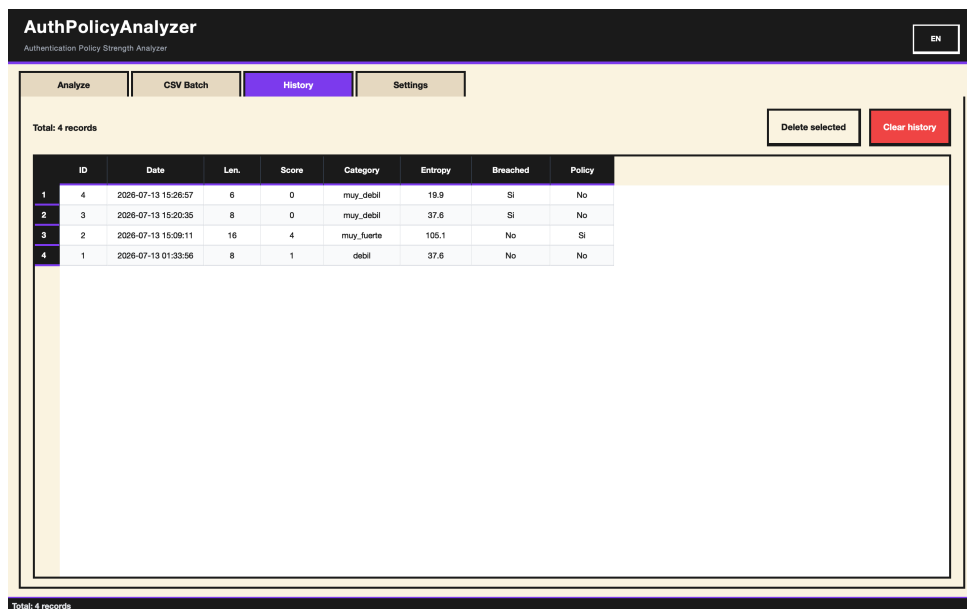


Figura 12: Interfaz — Pestaña Historial

Fuente: Elaboración propia

7.4. Pestaña Configuración

Permite ajustar la política de contraseñas (longitud, score, entropía, tipos de carácter) y el idioma, con botones para guardar o restaurar los valores predeterminados.

The screenshot shows the 'AuthPolicyAnalyzer' application interface, specifically the 'Configuración' (Configuration) tab. The interface has a dark header with the application name and a language selector set to 'ES'. Below the header is a navigation bar with four tabs: 'Analizar', 'Lote CSV', 'Historial', and 'Configuración' (which is active). The main content area is titled 'Configuración de Políticas' and contains two sections. The first section, 'Configuración de Políticas', includes three input fields: 'Longitud mínima (caracteres)' set to 12, 'Score mínimo aceptable (0-4)' set to 3, and 'Entropía mínima (bits)' set to 40.00 bits. Below these are four checkboxes, all of which are checked: 'Requiere mayúsculas', 'Requiere minúsculas', 'Requiere dígitos', and 'Requiere símbolos'. The second section, 'Idioma de la interfaz', has a dropdown menu for 'Idioma de la interfaz' currently set to 'Español (es)'. At the bottom right of the configuration area are two buttons: 'Restaurar predeterminados' and 'Guardar configuración'. A status bar at the very bottom indicates 'Idioma cambiado a: ES'.

Figura 13: Interfaz — Pestaña Configuración

Fuente: Elaboración propia